

编译原理

4. 抽象语法

rainoftime.github.io

**浙江大学
计算机科学与技术学院**

课程内容

1. Introduction
2. Lexical Analysis
3. Parsing
- 4. Abstract Syntax**
5. Semantic Analysis
6. Activation Record
7. Translating into Intermediate Code
8. Basic Blocks and Traces
9. Instruction Selection
10. Liveness Analysis
11. Register Allocation
13. Garbage Collection
14. Object-oriented Languages
18. Loop Optimizations

Recap

- Lexer
 - Program source to token sequence
- Parser
 - token sequence, and answer Y or N
- Today's (main) topic:
 - **Abstract syntax trees**

本讲内容

1

Attribute Grammar (不要求掌握)

2

Semantic Action

3

Abstract Parse Tree

1. Attribute Grammar

⚠ 本节不要求掌握，理解思想即可

编程语言 = 语法 + 语义

- **语法**: What sequences of characters are valid programs?
- **语义**: What is the behavior of a valid programs?
 - **操作语义**: How can we execute a program?
 - **公理语义**: What can we prove about a program?
 - **指称语义**: What math function does it compute?

The job of a **compiler** is to translate from the syntax of one language to another but **preserve the semantics**.

属性文法(Attribute Grammar) (Knuth, 1968)

- **属性文法: 上下文无关文法+属性+属性计算规则**
 - **属性:** 描述文法符号的语义特征如变量类型、值
例: 非终结符E的属性E.val (表达式的值)
 - **属性计算规则(语义规则):** 与产生式相关联、反映文法符号属性间关系的规则, 比如”如何计算E.val”

用**属性**描述语义信息, 用**语义规则**描述属性之间的关系, 将**语义规则与语法规则相结合**

属性文法(Attribute Grammar)的潜在应用

- “**推导类**”应用: 类似程序分析
 - 表达式的类型、作用域、值、甚至执行代价!

产生式	“语法制导”的属性计算规则
$E \rightarrow E_1 \text{ '+' } E_2$	$E.val := E_1.val + E_2.val$
$E \rightarrow E_1 \text{ '*' } E_2$	$E.val := E_1.val * E_2.val$
$E \rightarrow \text{'(' } E_1 \text{ ')'}$	$E.val := E_1.val$
$E \rightarrow \text{number}$	$E.val := \text{number.lex_val}$

- “**生成类**”应用: 类似程序合成
 - 抽象语法树生成、中间代码甚至汇编生成!

由于一些局限性，并未在所有潜在应用上得到普及

属性文法(Attribute Grammar)的实现

- **属性文法: 上下文无关文法+属性+属性计算规则**
 - **属性:** 描述文法符号的语义特征, 如变量的类型
例: 非终结符E的属性E.val (表达式的值)
 - **属性计算规则(语义规则):** 与产生式相关联、反映文法符号属性之间关系的规则, 如"怎么计算E.val"
- **具体实现: 可以通过Parser生成器支持的“语义动作”(Semantic action)实现**
 - 应用于抽象语法树生成、中间表示生成等场景

2. Semantic Action

Why Semantic Action

- 每个终结符和非终结符都可以关联自己的**语义值**
(semantic value)

$$A \rightarrow B C D$$

- 语义动作必须返回一个值，其**类型**与A关联的类型一致
- 它可以由B、C、D的语义值来构建 A 的结果

示例： $T \rightarrow T * F$

语义动作 `int a = T(); eat(TIMES); int b = F(); return a * b;`

Example: CFG and its Semantic Actions

- 目标：在语法分析的同时**计算**算术表达式的值

产生式	语义动作（属性计算规则）
$E \rightarrow E_1 + T$	$\{ E.val = E_1.val + T.val \}$
$E \rightarrow E_1 - T$	$\{ E.val = E_1.val - T.val \}$
$E \rightarrow T$	$\{ E.val = T.val \}$
$T \rightarrow (E)$	$\{ T.val = E.val \}$
$T \rightarrow \text{num}$	$\{ T.val = \text{num.val} \}$

此处语义值的类型是 `int`，语义规则描述了如何从子节点的值计算父节点的值。整个解析完成后，起始符号 `E` 的 `E.val` 就是表达式的最终求值结果。

Semantic Actions in Recursive Descent Parsing

- 语义动作可能是
 - 解析函数的**返回值**
 - 这些函数的**副作用**
 - 或**两者结合**
- 为每个终结符和非终结符关联一个**语义值类型**，表示从该符号推导出的短语所对应的语义信息

$T \rightarrow T * F$

The semantic action:

```
int a = T();  
eat(TIMES);  
int b=F();  
return a*b;
```

Semantic Actions in Yacc/Bison

- More declarative than the ones in top-down parsing
- `{ ... }`: 语义动作代码块
- `$i`: 第 i 个 RHS 符号的语义值
- `$$`: LHS 非终结符的语义值 (结果)
- `%union`: 声明语义值的可能类型集合
- `<variant>`: 声明每个符号的具体类型

```
%{ ... %}  
%union {int num; string id;}  
%token <num> INT  
%token <id> ID  
%type <num> exp  
...  
%left UMINUS  
%%  
  
exp: INT {$$ = $1;}  
    | exp PLUS exp {$$ = $1 + $3;}  
    | exp MINUS exp {$$ = $1 - $3;}  
    | exp TIMES exp {$$ = $1 * $3;}  
    | MINUS exp %prec UMINUS {$$ = -$2;}
```

Semantic Actions in Yacc-Generated Parsers

- Yacc 生成的解析器维护一个与**状态栈**并行的**语义值栈 (semantic value stack)**
- 当**归约 (reduce)**时，必须执行对应的 C 语言语义动作
- 对于规则 $A \rightarrow Y_1 \dots Y_k$ ， $\$i$ 如何获取？—— 来自**栈顶 k 个元素**
 - 从符号栈弹出 $Y_k \dots Y_1$ ，压入 A
 - 同时从**语义值栈弹出 k 个值**，执行 C 语义动作后，将结果 **$\$ \$$** 压入语义值栈

归约前（栈顶）

状态栈：

... s_1 s_2 s_3 $s_4 \leftarrow \text{top}$

语义值栈：

... v_1 v_2 $\$1$ $\$2 \leftarrow \text{top}$

归约后（ $A \rightarrow Y_1 Y_2$ ）

状态栈：

... s_1 s_2 $s_A \leftarrow \text{top}$

语义值栈：

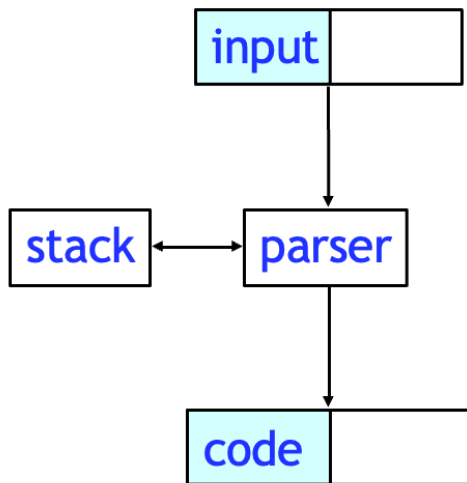
... v_1 v_2 $\$ \$ \leftarrow \text{top}$

2. Abstract Syntax Tree

- **AST是什么**
- **AST的构造**

Motivation

- In principle, we can write an entire compiler that fits **within the semantic action phrases** of a parser!
 - i.e., semantic actions generate code during parsing!
 - Especially for stack machine

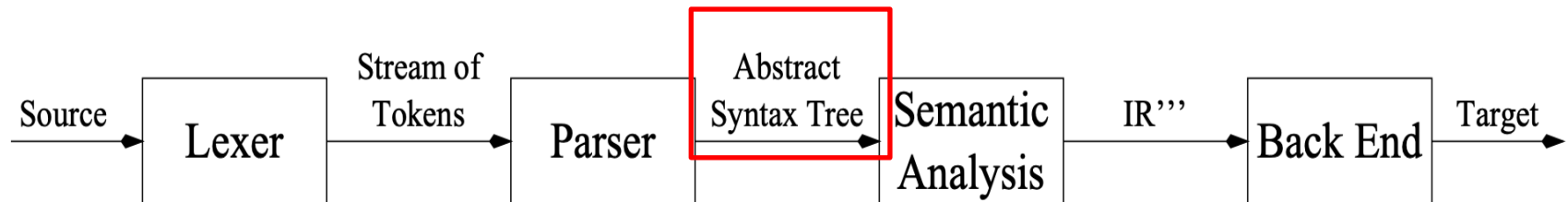


`expr ::= expr PLUS expr`
`{: emitCode(add); :}`

- Problems:
 - hard to maintain
 - limits language features
 - bad code!

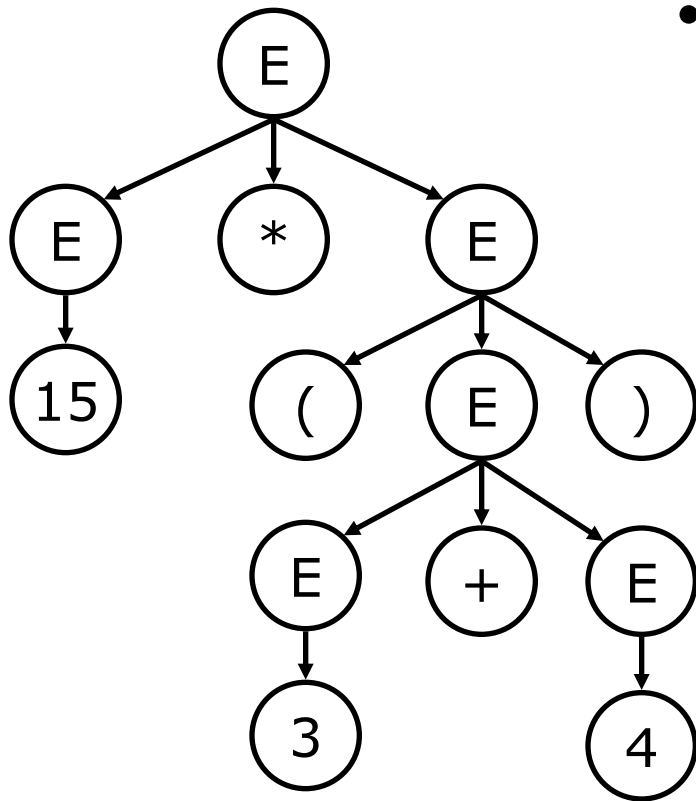
Motivation

- In principle, we can write an entire compiler that fits **within the semantic action phrases** of a parser!
- **Alternative:** separate issues of syntax (parsing) from issues of semantics
- One possible solution: the parser produces a **parse tree** that later phases can traverse?



Recap: (Concrete) Parse Trees

Parse trees encodes the grammatical structure of the source

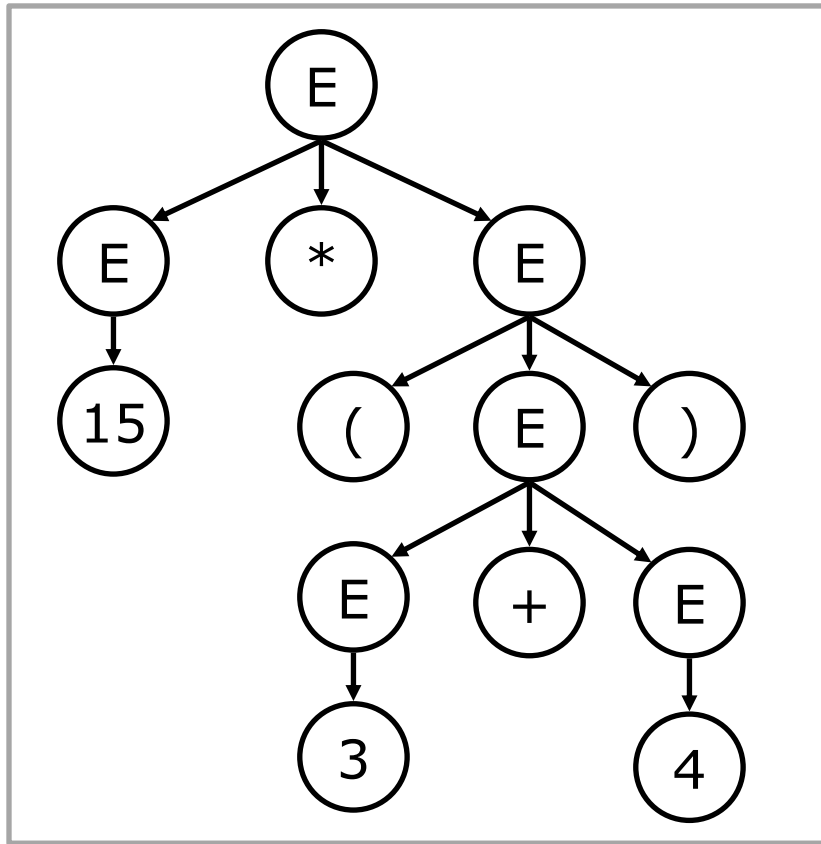


- A lot of unnecessary information
 - Redundant and useless tokens for later phases
 - Depends too much on the grammar
 - The grammar changes -> parse tree changes

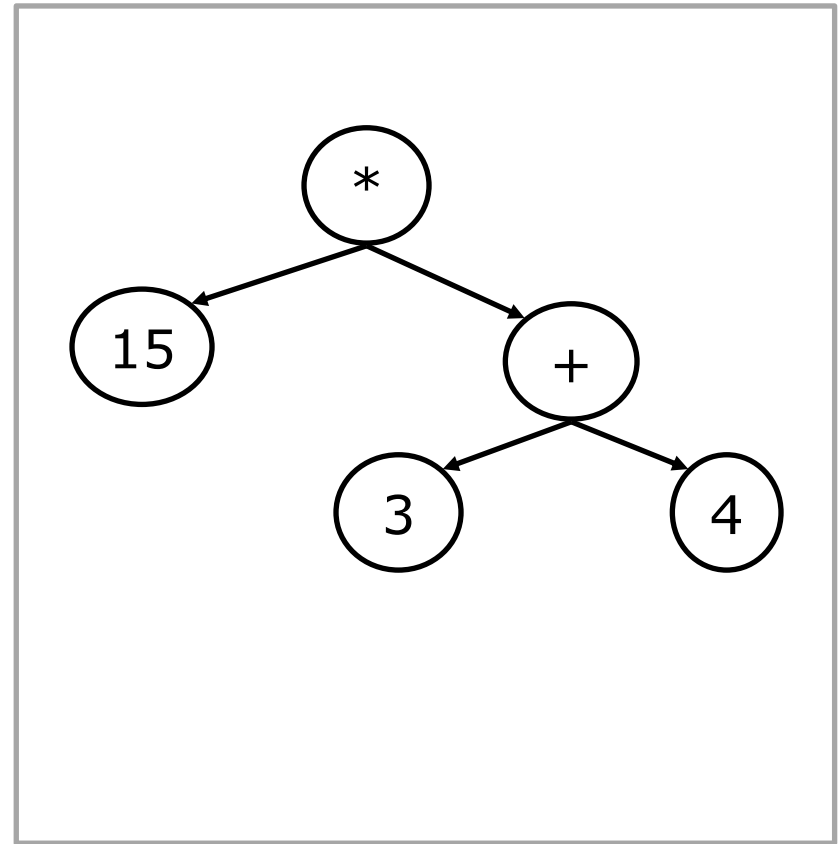
What are essential here?

Abstract Syntax Trees

- A clean interface between the parser and the later phases of a compiler.



具体语法树 *Parse tree*

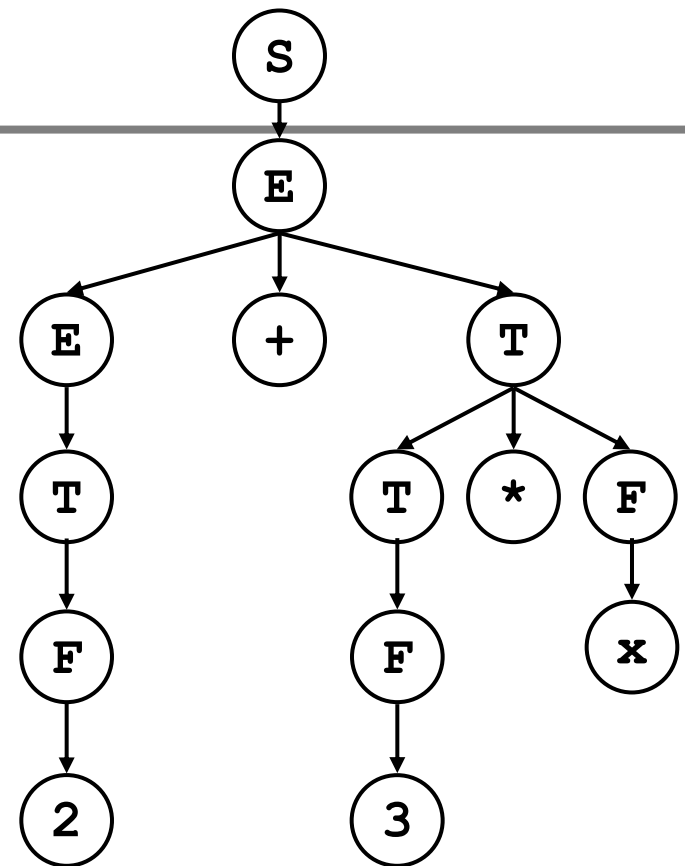


抽象语法树 *Abstract syntax tree*

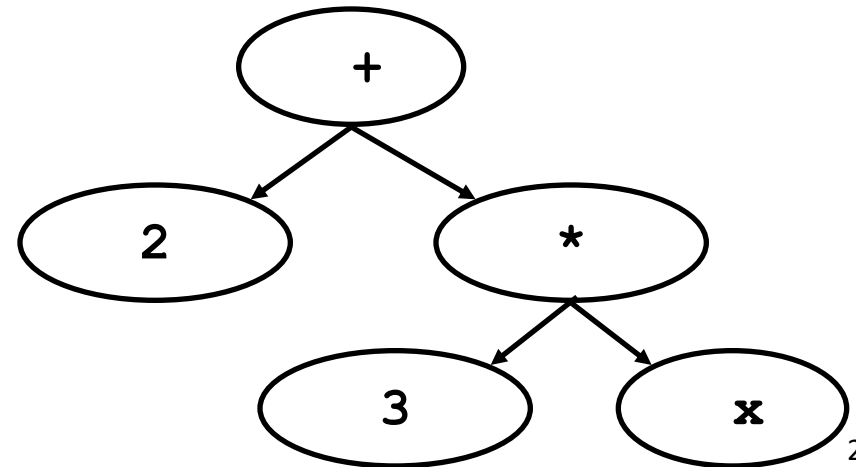
Example

```
E ::= E + T
    | T
T ::= T * F
    | F
F ::= id
    | num
    | ( E )
```

2 + 3 * x



```
E ::= id
    | num
    | E + E
    | E * E
    | ( E )
```



Applications of Abstract Syntax Trees (AST)

- **Applications inside compilers**
 - Semantic analysis , e.g., type checking
 - Translation to intermediate representations
 - Some high-level optimizations, e.g., constant fold
- **Applications outside compilers**
 - Pretty print, code editing, code diff, etc.
 - Analysis tools (e.g., Clang Static Analyzer)

注意: 编译相关书籍/课程中通常会说 “**基于AST翻译到中间语言/表示**”. 但在很多其他场合, 也可以认为AST本身也是一种 “中间表示”

2. Abstract Syntax Tree

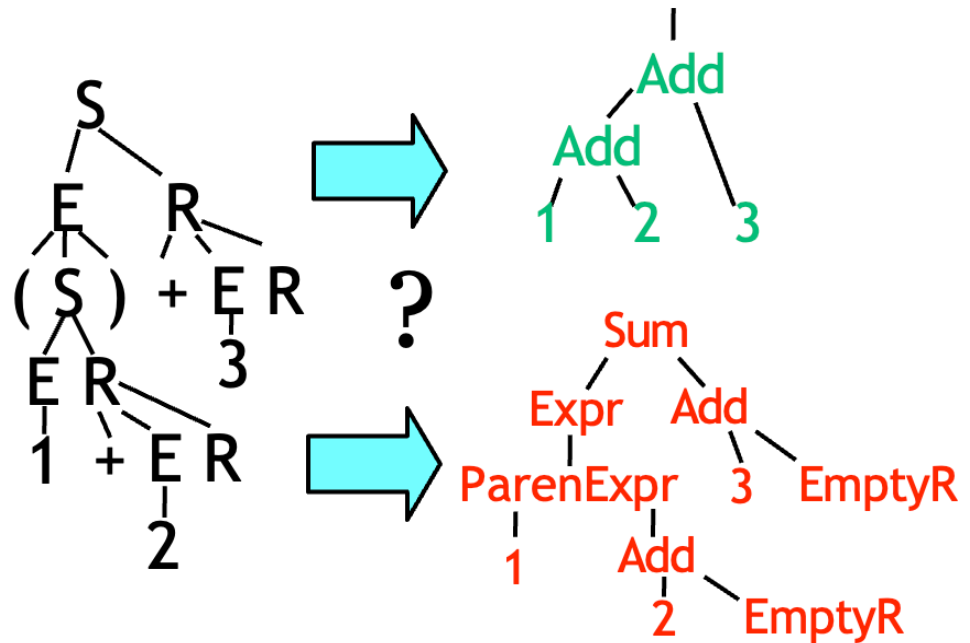
- AST是什么
- **AST**的表示
- AST的构造
 - Bottom-up
 - Top-down

How to Build the AST

- Bottom-up vs. top-down parsing
- AST can be built along parsing (e.g., via semantic actions)

$S \rightarrow E R$
 $R \rightarrow \varepsilon \mid + E R$
 $E \rightarrow \text{num} \mid (S)$

(1 + 2) + 3



AST for “Exp” in C

```
/* data structures */
typedef struct E *E;
enum kind {ID, INT, ADD, TIMES};
struct E {
    enum kind kind;
    union {
        char *id;
        int num;
        struct {E e1; E e2;} add;
        struct {E e1; E e2;} times;
    } u;
};
```

// This technique is called **tagged-union**.

对应的抽象文法

```
E ::= id
    | num
    | E + E
    | E * E
    | ( E )
```

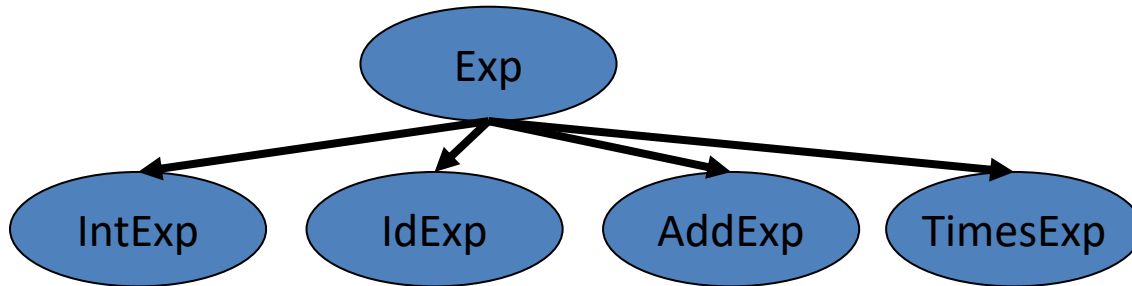
Tagged Union: 用枚举 kind 标记节点类型，用 union 存储各类型数据。C 语言中实现代数数据类型的经典方法。

AST for “Exp” in C: Operations are Tree-walkings

```
(* pretty printing *)
int pp (E e) {
    switch (e->kind) {
        case INT: printf ("%d", e->u.num); return;
        case ID:  printf ("%s", e->u.id); return;
        case ADD:
            printf ("("); pp (e->u.add.e1); printf (")");
            printf (" + ");
            printf ("("); pp (e->u.add.e2); printf (")");
            return;
        case TIMES: /* similar */
        default: error ("compiler bug");
    }
}
```

类型检查、代码生成等语义处理都遵循同样的模式：递归遍历 AST，
在每个节点做相应处理

AST for “Exp” in Java



```
/* data structures */  
abstract class Exp {}  
class IntExp extends Exp {  
    int i;  
    IntExp (int i){  
        this.i = i;  
    }  
}
```

```
// constructors omitted from the following classes
```

```
class IdExp extends Exp {String id;}  
class AddExp extends Exp {Exp e1;  
                           Exp e2;}  
class TimesExp extends Exp {Exp e1;  
                             Exp e2;}
```

```
E ::= id  
    | num  
    | E + E  
    | E * E  
    | ( E )
```

AST for “Exp” in F#

```
(* data structures *)
```

```
type E
```

```
  = Int of int
```

```
  | Id of string
```

```
  | Add of E * E
```

```
  | Times of E * E
```

```
(* to encode “2+3*x” *)
```

```
val prog = Add (Int 2
```

```
                , Times (Int 3
```

```
                        , Id “x”))
```

```
(* Easy and happy! *)
```

```
E ::= id  
    | num  
    | E + E  
    | E * E  
    | ( E )
```

函数式语言的代数类型与文法产生式直接对应，表达式直观简洁！

2. Abstract Syntax Tree

- AST是什么
- AST的表示
- **AST的构造**
 - **Top-down**
 - **Bottom-up**

AST Generations

- The “attribute-grammar scheme”
 - Each nonterminal may have a semantic value v associated with it
 - When the parser recognizes rule $X \rightarrow s_1 \dots s_n$
 - a semantic action will be executed
 - uses semantic values from symbols in s_i
 - When parsing completes successfully
 - Parser returns semantic value associated with the start symbol
 - Usually an abstract syntax tree

Creating the AST During Recursive-Descent Parsing

- 只需在每个解析函数中加入**构建 AST 节点**的代码！
- 可行原因：**解析树与递归调用树形状相同**——函数返回时自然地自底向上构建了 AST
- S, S', E 全部改为返回 Expr

void E() $\Rightarrow$ **Expr** E()

void S() $\Rightarrow$ **Expr** S()

void S'() $\Rightarrow$ **Expr** S'()

不再只是“识别”程序（返回 Yes/No），而是在识别的同时**构建**出代表程序结构的数据对象（AST）

2. Abstract Syntax Tree

- AST是什么
- AST的表示
- **AST的构造**
 - Top-down
 - **Bottom-up**

AST Generation in LR Parser

- The Yacc (or recursive-descent) parser, parsing the concrete syntax, constructs the abstract syntax tree.

```
%left PLUS
%left TIMES

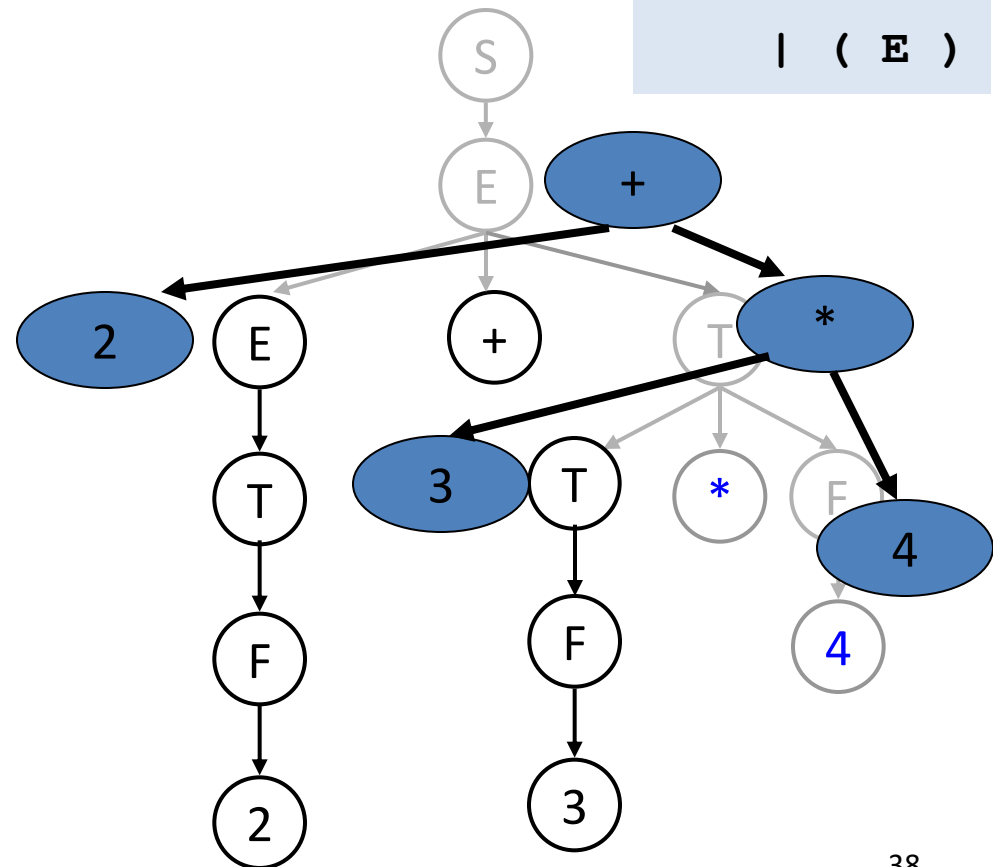
%%
exp : NUM                {$$=A_NumExp($1);}
    | exp PLUS exp       {$$=A_PlusExp($1, $3);}
    | exp TIMES exp      {$$=A_TimesExp($1, $3);}
```

- 语义值的类型：这里 `exp` 的语义值类型是 AST 节点指针（如 `A_exp`），而不是 `int`。
- \$1, \$3**：分别是左右 `exp` 的语义值，即左右子树的 AST 根节点。

Example: AST Generation in LR Parser

```
E ::= E + T
      | T
T ::= T * F
      | F
F ::= id
      | num
      | ( E )
```

	2 + 3 * 4
2	● + 3 * 4
F	● + 3 * 4
T	● + 3 * 4
E	● + 3 * 4
E +	● 3 * 4
E + 3	● * 4
E + F	● * 4
E + T	● * 4



Each nonterminal is associated with a tree.

Summary

- (Attribute grammar) and semantic actions
- Abstract syntax trees are compiler internal data structures of source programs
 - interface between front-end and compiler later parts
- Abstract syntax trees design is language-dependent
 - more art than science

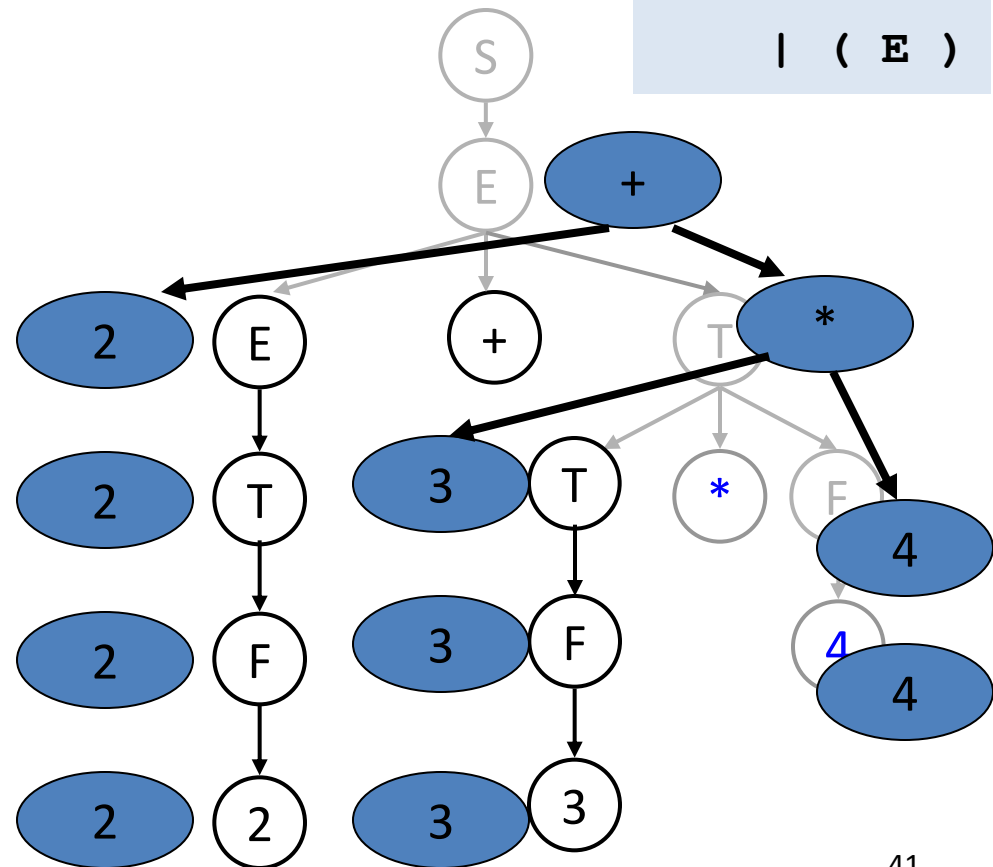


AST Generation in LR Parser

```
E ::= E + T
      | T
T ::= T * F
      | F
F ::= id
      | num
      | ( E )
```

	2 + 3 * 4
2	● + 3 * 4
F	● + 3 * 4
T	● + 3 * 4
E	● + 3 * 4
E +	● 3 * 4
E + 3	● * 4
E + F	● * 4
E + T	● * 4

Each nonterminal is associated with a tree.



4. Position

Positions

- In a one-pass compiler, lexical analysis, parsing, and semantic analysis are all done simultaneously.
- If there is a **type error** that must be reported to the user, the *current position* of the lexical analyzer is a reasonable approximation of the source position of the error.
- In a **one-pass** compiler, the lexical analyzer keeps a “**current position**” global variable.
- For a compiler that **uses abstract-syntax-tree data structures**:
 - It need not do all the parsing and semantic analysis in one pass
 - The lexer reaches the end of file before semantic analysis even begins.
- How can we know the error position if there is a type error?

Positions

- The source-file position of each node of the abstract syntax tree must be remembered.
- How?
- The abstract-syntax data structures must be sprinkled with **pos** fields, which indicate the **position**, within the original source file, of the characters **from which these abstract syntax structures were derived**.
- How to set the pos fields?
- First, the **lexer** must pass the positions of the beginning and end of each token to the **parser**
- Then, for parsers:

Positions

- Ideally, the parser should maintain a *position stack* along with the *semantic value stack*, making the position of each symbol available for the semantic actions to use.
 - Bison can do this, **Yacc does not**
- **Yacc**: one solution is to define a nonterminal symbol **pos** whose semantic value is a **source location** (line number, or line number and position within line).
 - e.g., access the position of the **PLUS**

```
%{ extern A_OpExp (A_exp,A_binop,A_exp,position); %}  
%union { int num; string id; position pos;...};  
%type <pos> pos  
  
pos: { $$ = EM_tokpos; }  
exp: exp PLUS pos exp {$$= A_OpExp($1, A_plus, $4, $3); }
```

一个递归下降的例子

(仅供参考 , 不要求掌握)

AST Generations: Recursive-Descent Parsing

- 原理：解析树与调用树形状相同，因此可以在递归调用返回时自然地自底向上构建 AST 节点

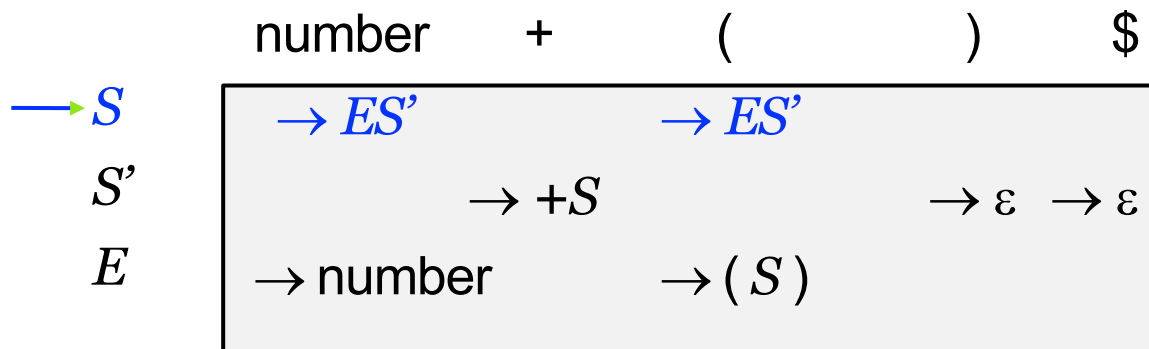
	num	+	(	)	\$
S	$\rightarrow ES'$			$\rightarrow ES'$	
S'		$\rightarrow +S$			$\rightarrow \epsilon \rightarrow \epsilon$
E	$\rightarrow \text{num}$		$\rightarrow (S)$		

$S \rightarrow ES'$
 $S' \rightarrow \epsilon \mid +S$
 $E \rightarrow \text{num} \mid (S)$

Example: Recursive-Descent Parser

```
void S () {  
    switch (token) {  
        case num: E(); S'(); return;  
        case '(': E(); S'(); return;  
        default: throw new ParseError();  
    }  
}
```

lookahead token



Example: Recursive-Descent Parser

```
void S'() {  
    switch (token) {  
        case '+': token = input.read(); S(); return;  
        case ')': return;  
        case EOF: return;  
        default: throw new ParseError();  
    }  
}
```

lookahead token

	number	+	(	)	\$
S	$\rightarrow ES'$		$\rightarrow ES'$		
$\rightarrow S'$		$\rightarrow +S$		$\rightarrow \epsilon$	$\rightarrow \epsilon$
E	$\rightarrow \text{number}$		$\rightarrow (S)$		

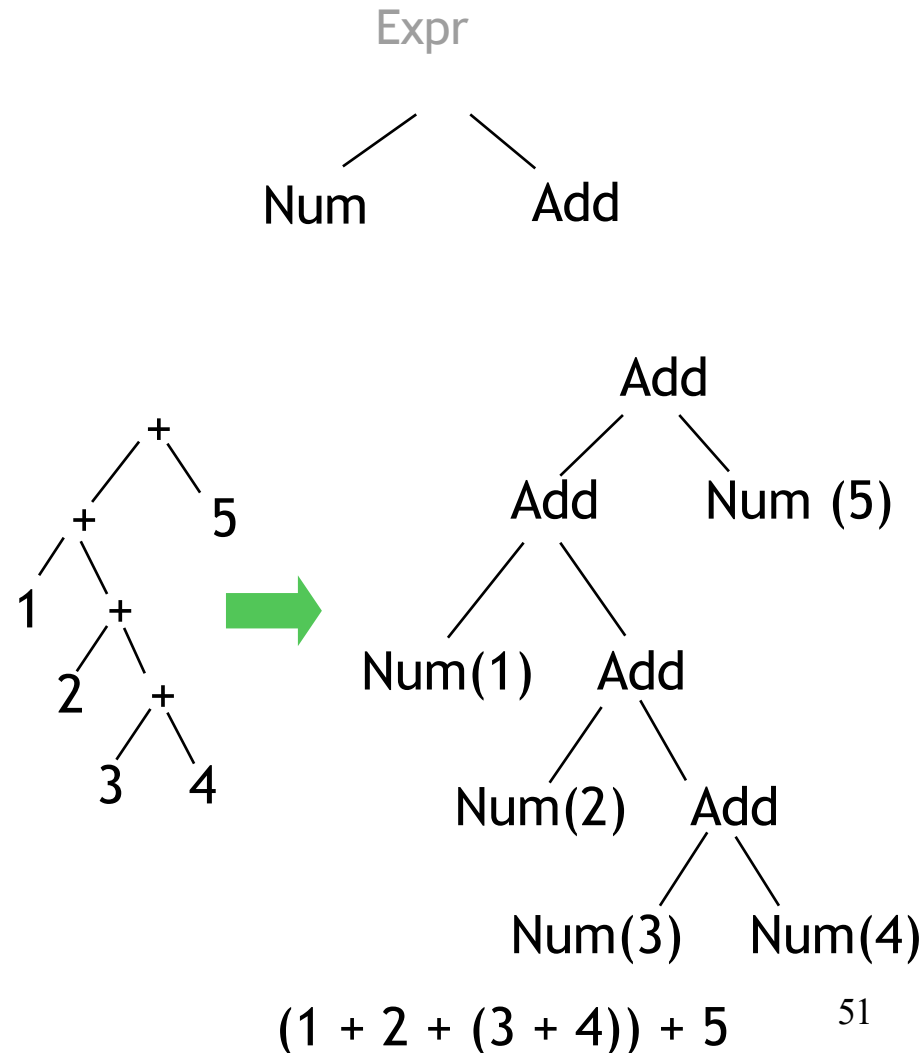
Example: Recursive-Descent Parser

```
void E() {  
    switch (token) {  
        case number: token = input.read(); return;  
        case '(': token = input.read(); S();  
            if (token != ')') throw new ParseError();  
            token = input.read(); return;  
        default: throw new ParseError(); }  
}
```

	number	+	(	)	\$
S	$\rightarrow ES'$		$\rightarrow ES'$		
S'		$\rightarrow +S$		$\rightarrow \varepsilon$	$\rightarrow \varepsilon$
$\rightarrow E$	$\rightarrow \text{number}$		$\rightarrow (S)$		

Representing the AST (In Java)

```
abstract class Expr { }  
class Add extends Expr {  
    Expr left, right;  
    Add(Expr L, Expr R) {  
        left = L; right = R;  
    }  
}  
  
class Num extends  
    Expr { int value;  
    Num (int v) { value = v);  
}
```



Creating the AST: the E() Function

```
Expr E() {  
    switch(token) {  
        case num: // E → number  
            Expr result = Num (token.value);  
            token = input.read(); return result;  
        case '(': // E → ( S )  
            token = input.read();  
            Expr result = S();  
            if (token != ')') throw new ParseError();  
            token = input.read(); return result;  
        default: throw new ParseError();  
    }  
}
```